

CIDECODE HACKATHON 2026

CCITR – CID Karnataka | in association with PES University, Bengaluru

SYSTEM DESIGN REPORT

Multi-Accused Cross-Account Investigation Engine

Problem Statement 1 — Automated Bank Statement Analysis System

Team: SURVEY CORPS

Document Stage: Stage 1 — Requirements, System Design & Analysis Phase Specification

Prepared during the Mentoring Phase (05–25 June 2026), ahead of the 24-hour hackathon on 4–5 July 2026

Document Version 1.0 | June 2026

Table of Contents

Table of Contents 2

1. Introduction 4

 1.1 Background 4

 1.2 Proposed Solution 4

 1.3 Objectives 4

 1.4 Scope of This Document 5

 1.5 Position in the Development Lifecycle 5

2. Requirements 6

 2.1 Functional Requirements..... 6

 2.2 Non-Functional Requirements 6

3. System Architecture Overview 8

 3.1 Three Architectural Layers 8

 3.2 The Four Processing Phases 8

 3.3 High-Level Component Map 8

4. Module-Level Design 10

 4.1 Input Module 10

 4.2 Extraction Module..... 10

 Component 1 — File Type Detection 10

 Component 2A — Digital PDF Extraction (pdfplumber)..... 10

 Component 2B — OCR & Pattern Cleaning (Tesseract + Python) 10

 Component 3 — Structure Identification via LLM 10

 Component 4 — Row Standardisation 11

 Component 5 — Validation & Cleaning 11

 4.3 Analysis Module 11

 4.4 Reporting Module 11

 4.5 RAG Chatbot Module 11

5. Analysis Phase — Detailed Design 12

 5.1 Purpose & Design Philosophy 12

 5.2 Investigator-Guided Analysis: The Case Brief 12

 5.3 Dynamic Threshold Generation 12

 5.4 The 25 Detection Cases 14

 Category A — Graph-Based Analysis (NetworkX) 14

 Category B — Time-Based Analysis (pandas) 14

 Category C — Amount-Based Analysis (pandas)..... 15

 Category D — Counterparty Analysis (pandas) 16

 Category E — Narration Analysis (pandas + LLM)..... 16

 Category F — Statistical Anomaly Detection (scikit-learn) 17

- Category G — LLM-Driven Analysis 17
 - 5.5 LLM Explanation Layer..... 18
 - 5.6 Composite Suspicion Scoring..... 18
 - 5.7 Output of the Analysis Phase 18
- 6. Report Generation Design 20
 - 6.1 Audience-Based Report Structure 20
 - Section 1 — Executive Summary..... 20
 - Section 2 — Detailed Findings per Account..... 20
 - Section 3 — Technical Appendix 20
 - 6.2 Money-Flow Visualisation in the Report..... 20
 - 6.3 Export Formats..... 20
 - 6.4 Role of the LLM in Reporting..... 21
 - 6.5 Implementation Order for Reporting 21
- 7. RAG-Based Investigation Chatbot Design 22
 - 7.1 Purpose..... 22
 - 7.2 RAG Pipeline..... 22
 - 7.3 Query Categories 22
 - 7.4 Conversation Context..... 22
 - 7.5 Limitations 23
 - 7.6 Why This Is the Standout Feature 23
- 8. Technology Stack 24
- 9. Data Storage & Data Flow 25
 - 9.1 Storage Components 25
 - 9.2 End-to-End Data Flow 25
- 10. Security & Privacy Considerations 27
- 11. Scalability & Deployment Considerations..... 28
- 12. Implementation Roadmap 29
 - 12.1 Recommended Build Order 29
 - 12.2 Mapping to the Hackathon Timeline 29
 - 12.3 Synthetic Dataset Strategy 29
- 13. Team Structure & Roles..... 31
- 14. Conclusion 32

1. Introduction

1.1 Background

Financial cybercrime investigations in India are severely hampered by the manual effort required to analyse bank statements. CID investigators regularly receive statements from multiple suspects, across different banks, in inconsistent formats — PDF, scanned images, and Excel sheets. The current process requires manually reviewing thousands of transactions, which takes weeks and often misses critical hidden connections between accounts.

The core challenges this project addresses are:

- **Format Inconsistency** — every bank produces statements in a different layout, making unified manual analysis impractical.
- **Volume Overload** — cyber-fraud cases routinely involve hundreds of accounts and thousands of transactions.
- **Hidden Networks** — money-mule chains and hawala networks span multiple accounts and are invisible without cross-account analysis.
- **Evidence Preparation** — manually compiling a court-ready forensic report from raw findings is slow and error-prone.

1.2 Proposed Solution

The Multi-Accused Cross-Account Investigation Engine is an AI-powered platform that combines Optical Character Recognition (OCR), Large Language Models (LLMs), and Retrieval-Augmented Generation (RAG) to ingest bank statements of multiple suspects simultaneously, detect sophisticated fraud patterns, visualise the money-flow network as an interactive graph, and automatically generate court-ready forensic reports.

The guiding principle of the design is that every numeric or structural finding is produced by deterministic code (pandas, NetworkX, scikit-learn) so that it is reproducible and legally defensible, while the LLM is used only where it provides genuine value over code — reading the investigator's natural-language case brief, generating context-appropriate thresholds, and writing plain-English explanations of what the mathematics found. This separation is what makes the tool's output trustworthy enough for an investigation file.

1.3 Objectives

- Provide a single platform to ingest bank statements in PDF, Excel, CSV, DOCX and scanned-image formats, for multiple suspects, in one upload.
- Normalise inconsistent bank-statement formats into one standard transaction schema using OCR and LLM-assisted structure identification.
- Allow the investigator to guide the analysis through a natural-language 'case brief' describing the specific crime pattern under investigation.
- Detect 25 categories of fraud and money-movement patterns — round-tripping, layering, smurfing, hawala, mule chains, dormancy activation, and more — using a combination of deterministic algorithms and LLM reasoning.
- Visualise the entire suspect network as an interactive money-flow graph showing who sent money to whom, and where it ultimately accumulated.
- Generate court-ready PDF and Excel forensic reports containing an executive summary, detailed per-account findings, and a technical appendix.

- Provide a RAG-powered conversational assistant that allows the investigator to query the actual case transaction data in plain English.
- Reduce the end-to-end investigation time for a multi-account financial case from weeks to minutes.

1.4 Scope of This Document

This document represents Stage 1 of the project's development lifecycle: requirements gathering and system design, prepared before any implementation begins. It covers:

- Functional and non-functional requirements derived from the PS1 brief and the team's solution synopsis.
- The overall system architecture across three layers (presentation, application, data) and four processing phases (input, extraction, analysis, output).
- A fully detailed specification of the Analysis Phase — the 25 detection cases, the case-brief mechanism, dynamic thresholds, the LLM explanation layer, and the suspicion-scoring model — since this is the intelligence core of the system.
- The extraction pipeline, report-generation design, and RAG chatbot design.
- The technology stack, data storage design, security and scalability considerations, and the implementation roadmap to be followed during the mentoring phase and the 24-hour hackathon.

1.5 Position in the Development Lifecycle

This document is the first deliverable in the following development sequence:

1. Stage 1 — Requirements & System Design (this document).
2. Stage 2 — Extraction Module Development (file-type routing, PDF/OCR extraction, LLM structure identification, standardisation, validation).
3. Stage 3 — Analysis Engine Development (the 25 detection cases across NetworkX, pandas, scikit-learn and the LLM).
4. Stage 4 — Report Generation Module (PDF via ReportLab, Excel via openpyxl).
5. Stage 5 — RAG Chatbot Development (ChromaDB ingestion, semantic retrieval, conversational interface).
6. Stage 6 — Frontend Development & Integration (React, interactive network graph, end-to-end wiring with FastAPI).
7. Stage 7 — Testing with Synthetic Data (deliberately planted fraud patterns used as an accuracy benchmark).
8. Stage 8 — Hackathon Demo Preparation and Presentation.

2. Requirements

2.1 Functional Requirements

The functional requirements below are derived directly from the PS1 'Automated Bank Statement Analysis System' brief and the team's solution synopsis, refined through the design discussions captured in this document.

ID	Requirement	Description
FR-01	Multi-format ingestion	The investigator can upload bank statements in PDF (digital and scanned), Excel, CSV, DOCX, and image formats (JPG/PNG), covering multiple suspect accounts, in a single session.
FR-02	Case-brief input	The investigator can enter a free-text description of the suspected crime pattern. This text contextualises every subsequent analysis step.
FR-03	OCR processing	Scanned documents and photographed statements are converted to machine-readable text via OCR before any further processing.
FR-04	Field detection & standardisation	The system automatically identifies Date, Narration, Debit, Credit, Balance, Account ID and Bank Name regardless of source format, and converts every transaction into one standard schema.
FR-05	Data cleaning & validation	Duplicate transactions are removed, failed/reversed transactions are detected, balance consistency is verified, and rows that cannot be resolved are flagged for manual review.
FR-06	Fraud-pattern detection engine	25 detection cases across graph, time, amount, counterparty, narration, statistical and LLM-driven categories are executed on the standardised dataset.
FR-07	Round-trip & money-trail analysis	Cyclic money-flow detection and FIFO-based money-trail tracing are implemented exactly as mandated by the PS1 core requirements.
FR-08	Money-flow visualisation	An interactive node-edge graph of all accounts and transactions is rendered, with suspicious accounts, mule chains and convergence points highlighted.
FR-09	Composite suspicion scoring	Every analysed account receives a score from 0–100, derived from the weighted contribution of every detection case that triggered for it.
FR-10	Report generation	A single action generates a court-ready PDF (executive summary, detailed findings, technical appendix) and a corresponding Excel export.
FR-11	RAG-powered chatbot	The investigator can query the uploaded transaction data in natural language and receive answers grounded in the actual case data.
FR-12	Session isolation	Each case session maintains its own dataset, transaction graph, scores, and chatbot context, independent of any other case.

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	The system must process datasets of 1,00,000+ transactions spanning up to 23 months and multiple suspects within a few minutes.
NFR-02	Accuracy & Explainability	All 22 rule-based detection cases must be fully deterministic and reproducible, so that every flagged finding can be traced back to a specific calculation and withstand legal scrutiny.
NFR-03	Usability	The interface must be usable by a police investigator with no technical or data-analysis background; no command-line interaction should be required.

ID	Category	Requirement
NFR-04	Reliability	Rows that cannot be extracted or validated with confidence must be flagged for manual review rather than silently dropped or guessed.
NFR-05	Scalability	The architecture must allow the system to scale, with configuration changes to storage and deployment only, from a single-machine hackathon demo to a multi-office production deployment.
NFR-06	Security & Privacy	Although full implementation is out of scope for the hackathon, the design must account for encryption of sensitive financial data, authenticated access, and audit logging in production.
NFR-07	Maintainability	Extraction, analysis, reporting and chatbot must remain loosely coupled modules that can be developed, tested and replaced independently.
NFR-08	Cost Efficiency	LLM API calls must be minimised and cached. The design restricts LLM usage strictly to tasks where it provides genuine value over deterministic code.

3. System Architecture Overview

3.1 Three Architectural Layers

The system is organised into three layers, each with a distinct responsibility:

Presentation Layer (React frontend): Everything the investigator sees and interacts with — the upload and case-brief screen, the analysis progress screen, the results screen with score rankings, interactive network graph and download links, and the chatbot panel.

Application Layer (Python / FastAPI backend): All logic for the system lives here. FastAPI exposes a small set of REST endpoints and internally runs the extraction pipeline, the analysis engine, the report generator and the RAG chatbot pipeline.

Data Layer: Three distinct storage mechanisms — an in-memory pandas dataframe for the live working dataset, ChromaDB on disk for the chatbot's searchable vector store, and local file storage for uploaded files, generated reports and cached LLM responses. Section 9 covers this layer in full detail.

3.2 The Four Processing Phases

Every case processed by the system moves through four sequential phases. The RAG chatbot operates as a fifth, parallel interface that becomes active once Phase 4 completes.

Phase 1 — Input: The investigator types the case brief and uploads all suspect statements. FastAPI saves the files and stores the case brief in session memory.

Phase 2 — Extraction: Every uploaded file, regardless of format, is converted into one standardised transaction table (detailed in Section 6).

Phase 3 — Analysis: The 25 detection cases run against the standardised table, guided by the case brief, producing findings and suspicion scores (detailed in Section 5).

Phase 4 — Output: The PDF and Excel forensic reports are generated, the interactive graph and score rankings are sent to the frontend, and the chatbot becomes available (detailed in Sections 6 and 7).

3.3 High-Level Component Map

Layer	Component	Primary Technology	Purpose
Presentation	Upload & Case Brief Screen	React, TailwindCSS	Lets the investigator type the case brief and upload all suspect statements in one action.
Presentation	Analysis Progress Screen	React + polling/websockets	Shows live status while extraction and analysis run on the backend.
Presentation	Results Screen	React, Cytoscape.js / D3.js	Displays suspicion-score rankings, the interactive money-flow graph, findings, and download links.
Presentation	Chatbot Panel	React chat UI	Lets the investigator ask natural-language follow-up questions after the report is ready.
Application	API Layer	FastAPI, uvicorn	Exposes /upload, /status, /results, /report, /export and /chat endpoints; orchestrates every phase.

Layer	Component	Primary Technology	Purpose
Application	Extraction Pipeline	pdfplumber, Tesseract, pandas, LLM	Converts any uploaded file into one standardised transaction table.
Application	Analysis Engine	pandas, NetworkX, scikit-learn, LLM	Runs the 25 detection cases and produces findings and suspicion scores.
Application	Report Generator	ReportLab, openpyxl, matplotlib	Assembles the PDF and Excel forensic reports from the analysis findings.
Application	RAG Chatbot Pipeline	ChromaDB, sentence-transformers, LLM	Answers natural-language questions grounded in the case's actual transactions.
Data	In-Memory Dataframe	pandas (RAM)	Holds the clean transaction table and analysis results for the duration of the session.
Data	Vector Store	ChromaDB (disk)	Holds every transaction as a searchable embedding for the RAG chatbot.
Data	Local File Storage	Disk — uploads / outputs / cache folders	Holds uploaded files, generated reports, and cached LLM responses.

4. Module-Level Design

4.1 Input Module

The landing screen presents the investigator with two elements: a case-brief text area and a multi-file drag-and-drop upload zone accepting PDF, Excel, CSV, DOCX, JPG and PNG files. When the investigator clicks Analyse, the React frontend sends both the case-brief text and all uploaded files to the FastAPI /upload endpoint as a single multipart request.

FastAPI saves every uploaded file to the local uploads folder, stores the case-brief text in session memory, assigns a session ID, and immediately triggers the extraction pipeline for that session. The session ID is returned to the frontend, which uses it to poll the /status endpoint while processing continues in the background.

4.2 Extraction Module

The extraction module's single job is to take any combination of PDF, Excel, CSV, DOCX or image files and convert all of them into one unified transaction table with the same columns every time — Date, Narration, Debit, Credit, Balance, Account ID and Bank Name. It is built from five components that run in sequence, like an assembly line.

A key design point is how the module scales to large datasets — for example, ten suspects with 23 months of statements each, totalling over 1,00,000 transaction rows. The LLM (Component 3) only ever sees the first 30–40 lines of each document, so the number of LLM calls equals the number of documents, not the number of rows. Every remaining row, however many there are, is processed by pandas, which handles a few lakh rows in seconds. Large volume is therefore never a bottleneck — it simply means more rows flowing through Components 4 and 5.

Component 1 — File Type Detection

The first step for every uploaded file is a simple check of its extension and MIME type. Based on this, the file is routed to one of the downstream extraction paths: Excel/CSV files go directly to Component 4; digital PDFs go to Component 2A; images and scanned PDFs go to Component 2B first.

Component 2A — Digital PDF Extraction (pdfplumber)

For PDFs generated by a computer (i.e., the text is stored digitally inside the file), pdfplumber reads every page and extracts the embedded text directly. All pages are combined into one continuous text block, which is passed to Component 3.

Component 2B — OCR & Pattern Cleaning (Tesseract + Python)

For scanned PDFs and photographed statements, there is no digital text inside the file — only pixels. Tesseract OCR converts these pixels into raw text. Because this raw text is often messy (misaligned columns, numbers and dates running together), a Python pattern-matching pass runs immediately afterwards: anything matching a date pattern (DD/MM/YYYY) is tagged as a date, anything matching a number-with-commas pattern is tagged as an amount, and everything else is treated as narration. This tagged, cleaned text is passed to Component 3.

Component 3 — Structure Identification via LLM

At this point the system has a block of text containing transaction data, but does not yet know which part is the date, which is debit, which is credit, and which is balance — because every bank's layout differs. The first 30–40 lines of the text are sent to the LLM with a single question: what is the column order here? The LLM returns a small JSON object describing the column positions. This call happens exactly once per document — whether the document has 100 rows or 1,00,000 rows — and the response is cached locally so repeated test runs do not consume additional API quota.

Component 4 — Row Standardisation

Python takes over completely from this point. Using the column structure identified in Component 3 (or directly, for Excel/CSV files), every row is parsed into the standard columns. Dates are converted to one consistent format, commas are stripped from numeric values (so '90,000' becomes 90000 for calculation), and the account ID and bank name are attached to every row. The output is one unified pandas dataframe covering every uploaded document.

Component 5 — Validation & Cleaning

Three checks run on every row of the standardised table:

- Date validity — the date field must parse as a real date; a value like '90000' in the date column indicates a misread.
- Balance arithmetic — previous balance $\pm$ debit/credit must equal the current balance; a mismatch indicates a misread row.
- Debit/credit exclusivity — exactly one of the debit and credit fields should contain a value per row; both being populated indicates column misalignment.

Rows that fail any of these checks are moved to a separate 'flagged for manual review' list and shown to the investigator with a warning. Clean rows proceed into the analysis phase. This honest handling of imperfect OCR output is an intentional design choice — no system can claim 100% OCR accuracy, and surfacing uncertainty is itself part of a defensible process.

After validation, the clean table is written to two destinations simultaneously: the in-memory pandas dataframe used by the analysis engine, and ChromaDB, where each transaction is converted into a descriptive sentence and stored as a searchable vector for the RAG chatbot.

4.3 Analysis Module

The analysis module receives the standardised pandas dataframe together with the case brief, and runs the 25 detection cases described in full in Section 5. Its output is an enriched dataframe, a list of structured findings (each with an LLM-written explanation), and a ranked table of per-account suspicion scores.

4.4 Reporting Module

The reporting module consumes the analysis output and produces the two artefacts the investigator downloads: a court-ready PDF assembled by ReportLab, and a working Excel export assembled by pandas and openpyxl. Its full design is given in Section 6.

4.5 RAG Chatbot Module

The RAG chatbot module becomes active once Phase 4 completes. It allows the investigator to ask natural-language questions about the uploaded case data, using ChromaDB for semantic retrieval and the LLM for answer generation. Its full design is given in Section 7.

5. Analysis Phase — Detailed Design

The analysis phase is the intelligence core of the system. It receives the standardised transaction table produced by the extraction module, together with the investigator's case brief, and runs 25 detection cases organised into seven categories. This section is intentionally the most detailed in the document, since it is where the system's value — and its credibility in court — is created.

5.1 Purpose & Design Philosophy

Of the 25 detection cases, 22 are entirely deterministic — implemented in pandas, NetworkX and scikit-learn — and cannot hallucinate. The remaining 3 cases are the only ones where the LLM performs reasoning that affects which transactions get flagged (Section 5.4, Category G). Every other use of the LLM in the analysis phase is confined to explaining findings that the deterministic code has already produced.

This separation is the system's core design principle: the mathematics finds every pattern, and the LLM explains every pattern in language a senior officer or a court can understand. Together, this combination is intended to produce a report that is both comprehensive and legally defensible, in two to three minutes instead of two to three weeks.

5.2 Investigator-Guided Analysis: The Case Brief

In real-world practice, an investigating officer collects bank statements and writes a 'brief' for a forensic auditor — a short note describing what is suspected, for example: the suspect appears to be receiving small payments from many different people. The auditor then manually searches for exactly that pattern. This tool replicates that exact workflow, except the 'auditor' is the analysis engine, and the search happens in minutes rather than days.

The case-brief text box on the input screen is therefore not just a label — it becomes part of the system prompt sent to the LLM for every analysis call. A representative system prompt is structured as follows:

```
system_prompt = f"""
You are a financial fraud analyst assisting CID Karnataka police.
Case brief from investigator: {case_brief}
Analyse the transactions below based on this specific context.
Flag transactions that match the pattern described.
"""
```

An example case brief, as the investigator might type it, would read:

“Suspect is allegedly receiving payments for drug supply. Amounts are typically between Rs 2,000 and Rs 5,000. Payments come from multiple different people. The suspect then transfers the accumulated amount to a single account within 24 hours.”

With this context, the LLM-driven cases (Category G) are no longer doing generic fraud detection — they are searching for precisely the pattern the investigator described, while the 22 deterministic cases run in parallel using thresholds tuned to that same pattern (Section 5.3). This case-brief mechanism is the single feature that most clearly differentiates an investigator-guided analysis system from a generic bank-statement scanner.

5.3 Dynamic Threshold Generation

Before the 22 deterministic cases execute, the case brief is sent to the LLM once, and the LLM returns a structured set of thresholds — for example, the amount band for structuring detection, the dwell-time limit that defines a 'pass-through' account, the velocity multiplier that counts as a spike, and the time windows used for synchronisation and hawala matching. These values replace the system's default thresholds for that session only. This is what allows the same 22 algorithms to adapt to any crime type the investigator describes — a drug-payment case, a loan-app harassment case, or a government-

scheme fraud case — without any code changes. This mechanism is itself Detection Case 23, described in full in Section 5.4, Category G.

5.4 The 25 Detection Cases

The table below documents every detection case: what it is looking for (Definition), how it is computed (Detection Approach), and whether it is implemented purely in code, purely via the LLM, or as a hybrid of the two (Type).

Category A — Graph-Based Analysis (NetworkX)

These five cases construct a single directed transaction graph — accounts as nodes, transactions as edges carrying amount and timestamp — and apply graph-theory algorithms to surface network-level structures that are invisible when statements are reviewed account-by-account.

#	Detection Case	Definition	Detection Approach	Type
1	Round-Trip Detection	Money leaves an account, passes through one or more intermediary accounts, and returns to the same account or a related account, forming a circular money flow (e.g., Victim → Accused 1 → Accused 2 → back to a related account).	All transactions are loaded into a directed graph. NetworkX's cycle-finding algorithm (<code>simple_cycles</code>) is run over the graph to mathematically enumerate every closed loop, regardless of how many hops it contains.	Hard-Coded
2	Multi-Hop Layering Trace	Funds move through a chain of accounts in a straight line — A to B to C to D — to distance the money from its origin, a classic money-laundering 'layering' technique.	Starting from a source node, the graph is traversed hop-by-hop along directed edges, recording the amount and timestamp at each step until a terminal (non-onward-transferring) account is reached.	Hard-Coded
3	Hub Account Identification	A single account acts as the central connector across multiple independent transaction chains — effectively the coordination point of the network.	Degree centrality and betweenness centrality are computed for every node in the transaction graph. Accounts whose centrality is statistically far above the network average are flagged as hubs.	Hard-Coded
4	Isolated Cluster Detection	A group of accounts transacts only among themselves and has no transactions with any account outside the group — a self-contained, closed network.	NetworkX's connected-components function partitions the overall transaction graph into sub-graphs. Any sub-graph with no edges leading to nodes outside it is flagged as an isolated cluster.	Hard-Coded
5	Convergence Point Identification	Multiple independent money chains, starting from different and unrelated sources, all terminate in the same destination account — indicating fund aggregation.	The graph is searched for terminal nodes whose incoming edges trace back to three or more distinct, non-overlapping source chains. Such nodes are flagged as convergence points.	Hard-Coded

Category B — Time-Based Analysis (pandas)

These five cases analyse the timing and rhythm of transactions using pandas datetime operations to expose behavioural signatures — such as mule pass-through speed, sudden activity, or scheduled criminal payments — that pure amount analysis cannot reveal.

#	Detection Case	Definition	Detection Approach	Type
6	Dwell Time Analysis	Measures how long money remains in an account between the moment it is credited and the moment it is next debited out. A near-zero dwell time is the signature of a pure pass-through 'mule' account.	For every credit transaction, the time gap to the next debit transaction in the same account is calculated. Accounts whose average dwell time falls below the case threshold are flagged.	Hard-Coded

#	Detection Case	Definition	Detection Approach	Type
7	Dormancy Activation Detection	An account that has been inactive for an extended period (six months or more) suddenly becomes active again, often with high-value transactions.	The gap between consecutive transaction dates is calculated per account. Any large gap immediately followed by a sudden, high-value transaction triggers a dormancy-activation flag.	Hard-Coded
8	Velocity Spike Detection	The frequency of transactions for an account suddenly increases five times or more above that account's own historical average.	A rolling-window transaction count is computed per account and compared against the account's historical baseline frequency using pandas' rolling-window functions.	Hard-Coded
9	Synchronized Transaction Detection	Multiple, otherwise unrelated accounts all carry out transactions within the same narrow time window (e.g., a 30-minute slot) on the same date — suggesting a single person or group is operating all of them.	Transactions across all uploaded accounts are grouped into fixed time buckets using pandas groupby. Buckets containing activity from several different accounts simultaneously are flagged.	Hard-Coded
10	Periodicity Detection	The same amount is sent to the same counterparty at regular, fixed intervals (e.g., every Monday, or every 15 days) — consistent with scheduled criminal payments such as drug supply instalments or extortion / protection money.	Transactions are grouped by account-counterparty-amount combinations, and the time differences between successive occurrences are checked for regularity using pandas date-difference operations.	Hard-Coded

Category C — Amount-Based Analysis (pandas)

These five cases focus on the values of transactions and how money is broken up, matched, or consumed — covering structuring, hawala-style transfers, the FIFO money trail mandated by the problem statement, cash-out behaviour, and collection patterns.

#	Detection Case	Definition	Detection Approach	Type
11	Structuring / Smurfing Detection	Many small credits, each just below a reporting threshold (e.g., between Rs 9,500 and Rs 9,999), arrive from a large number of unique senders — a deliberate technique to stay below automated reporting limits.	Transactions are filtered to the relevant amount band (band defined dynamically per case, see Section 5.3) and the count of unique senders per receiving account is computed. Accounts exceeding the unique-sender threshold are flagged.	Hard-Coded (band set by LLM)
12	Hawala Matched-Pair Detection	A debit of amount X from one account is matched by a credit of approximately X minus a small commission (2-3%) into a different account within a short time window — the commission difference being the hawala operator's cut.	Debit transactions in one account are cross-matched against credit transactions in other uploaded accounts that fall within a 48-hour window and within a 2-3% value tolerance of the original debit.	Hard-Coded
13	FIFO Money Trail Analysis	For every credit received by an account, traces exactly which subsequent debit transactions consumed that money, until the balance returns to its pre-credit level. This case is explicitly mandated by the PS1 core requirements.	A First-In-First-Out queue of unconsumed credits is maintained per account. Each subsequent debit is allocated against the oldest unconsumed credit first, producing a transaction-level 'where did the money go' trail.	Hard-Coded

#	Detection Case	Definition	Detection Approach	Type
14	Cash Withdrawal Fragmentation Detection	A large incoming credit is followed, often on the same day, by multiple smaller ATM withdrawals — frequently across different locations — converting traceable bank funds into untraceable cash.	Large credit transactions are identified, and the same day's subsequent debit transactions are scanned for ATM-related narration keywords. Multiple matching small withdrawals trigger the flag.	Hard-Coded
15	Micro-Transaction Aggregation Detection	Many small credits from many different senders accumulate in an account, which is then emptied via a single large outgoing transfer — the classic 'retail collection' pattern.	A rolling sum of small incoming credits over a defined window is compared against subsequent single large outgoing debits using pandas rolling and groupby operations.	Hard-Coded

Category D — Counterparty Analysis (pandas)

These four cases examine the relationships between an account and the other parties it transacts with — who it receives from, who it sends to, and how those relationships change over time — to surface mule-account and aggregation signatures.

#	Detection Case	Definition	Detection Approach	Type
16	Fan-In / Fan-Out Ratio Analysis	An account receives money from a very large number of unique senders but sends to very few receivers (or the reverse) — the textbook signature of a 'collector' or 'distributor' node, regardless of the underlying crime type.	The number of unique senders and unique receivers per account is computed using pandas groupby and nunique, and the ratio between the two is compared against a configured threshold.	Hard-Coded
17	Single Counterparty Concentration	An account sends 90% or more of its total outgoing value to a single destination account — the clearest possible signature of a pass-through mule account.	Total outgoing value per account is grouped by destination counterparty, and the percentage share of each destination is calculated. Any destination exceeding the configured concentration threshold is flagged.	Hard-Coded
18	Ghost Beneficiary Detection	Multiple different names appear in transaction narrations, all crediting the same underlying account number — a pattern commonly seen in government scheme fraud (e.g., MGNREGA-style misappropriation).	Narrations are grouped by destination account number, and the set of distinct beneficiary names associated with each account number is examined for multiplicity.	Hard-Coded
19	New Counterparty Spike Detection	An account that historically dealt with only three or four regular contacts suddenly transacts with fifty or more new accounts within a single month — indicative of account takeover or mule recruitment.	The set of counterparties in the current analysis period is compared against the historical baseline set for the same account using a pandas set-difference operation.	Hard-Coded

Category E — Narration Analysis (pandas + LLM)

These two cases analyse the free-text narration field attached to each transaction — first as a pure text-pattern problem, then as an investigator-guided keyword search powered by the LLM.

#	Detection Case	Definition	Detection Approach	Type
20	Blank / Generic Narration Clustering	A high proportion of an account's transactions carry blank, empty, or generic narrations such as 'transfer'	The percentage of blank or generic narrations is computed per account using simple pandas string-	Hard-Coded

#	Detection Case	Definition	Detection Approach	Type
		or 'payment'. Criminal transactions deliberately avoid descriptive narrations to reduce traceability.	matching and counting, and compared against a configured percentage threshold.	
21	Case-Brief Keyword Matching	Searches every transaction narration for specific names, keywords, or amount ranges that the investigator mentioned in the case brief — turning the investigator's own words into a search filter.	The LLM reads the case brief and extracts relevant keywords, names and amount ranges as structured output. Pandas then searches the narration field of every transaction for these extracted terms.	Hybrid (LLM extracts → rules search)

Category F — Statistical Anomaly Detection (scikit-learn)

This single case acts as the system's safety net — catching unusual transactions that do not match any of the 24 explicitly defined patterns above.

#	Detection Case	Definition	Detection Approach	Type
22	Isolation Forest Anomaly Detection	Identifies the most statistically unusual transactions in the entire uploaded dataset, without requiring any external training data, catching unknown or emerging patterns that the other 24 cases do not explicitly cover.	An Isolation Forest model is trained directly on the uploaded transactions, using features such as amount, hour of day, day of week, transaction frequency for the account, and balance ratio. The top ~5% most anomalous transactions are flagged.	Hard-Coded (self-trained ML)

Category G — LLM-Driven Analysis

These three cases are the only places in the entire analysis engine where the LLM performs reasoning that directly shapes which transactions get flagged, rather than simply explaining findings that code has already produced.

#	Detection Case	Definition	Detection Approach	Type
23	Dynamic Threshold Generation	Generates case-specific numeric thresholds — structuring amount bands, dwell-time limits, velocity multipliers, time windows — based on the investigator's case brief, instead of relying on one-size-fits-all default values.	The LLM reads the case brief once and returns a structured JSON object of thresholds. These values are substituted into the parameters of all 21 remaining rule-based cases before they execute.	LLM-Only
24	Case-Brief Pattern Matching	Extracts custom filter rules described by the investigator in free text that do not correspond to any of the 24 predefined cases, and applies them dynamically to the dataset.	The LLM converts the investigator's free-text description into structured filter rules (field, operator, value). Pandas then executes these rules as dynamic queries against the transaction dataframe.	Hybrid (LLM generates rules → pandas executes)
25	Hypothesis Generation (Blind-Audit Mode)	When the investigator provides no case brief, the system independently proposes which type of financial crime the detected pattern most closely resembles, with a confidence level and supporting reasoning.	The complete set of findings from the 22 hard-coded cases is sent to the LLM, which classifies the likely crime type (e.g., smurfing, hawala, mule network, layering) and explains the reasoning behind the classification.	LLM-Only

5.5 LLM Explanation Layer

Every detection case that triggers for an account produces a structured 'finding' object containing the account ID, the case type, the severity, the specific amounts and timestamps involved, and the relevant transaction IDs. Each finding object — together with the case brief — is sent to the LLM, which returns three things:

- A plain-English explanation of what was found, written for a non-technical reader.
- An evidence-strength rating of High, Medium or Low.
- A recommended next action for the investigator (e.g., 'cross-verify ACC009 with the suspect's KYC records').

These three outputs are stored alongside the finding and are inserted directly into the corresponding section of the forensic report (Section 6). This is the layer that turns 'Case 11 triggered for ACC001 with severity HIGH' into a sentence a senior officer can act on without needing to understand pandas or NetworkX.

5.6 Composite Suspicion Scoring

Every account that appears in the dataset receives a composite suspicion score from 0 to 100. The score is the sum of weighted points contributed by each of the 25 cases that triggered for that account. Cases that represent stronger, more specific evidence of deliberate fund movement contribute more points than cases that represent general behavioural signals. The table below illustrates the indicative weighting tiers; exact point values will be tuned during Stage 3 implementation and validated against the synthetic dataset described in Section 12.3.

Tier	Example Cases	Rationale
High evidence (20–25 pts each)	Round-Trip Detection, FIFO Money Trail, Hawala Matched Pair, Convergence Point Identification, Multi-Hop Layering Trace	These cases describe a specific, mathematically certain flow of funds and are the strongest possible evidence of deliberate movement.
Medium evidence (10–19 pts each)	Structuring/Smurfing, Single Counterparty Concentration, Dormancy Activation, Velocity Spike, Periodicity, Cash Withdrawal Fragmentation, Hub Identification, Isolated Cluster, Micro-Transaction Aggregation, Fan-In/Fan-Out, Ghost Beneficiary, New Counterparty Spike, Synchronized Transactions, Isolation Forest anomaly	These cases describe a strong behavioural pattern that is highly correlated with fraud but could, in isolated cases, have an innocent explanation.
Low / supporting evidence (3–9 pts each)	Dwell Time Analysis, Blank Narration Clustering, Case-Brief Keyword Matching	These cases provide supporting context that strengthens a case built primarily on the higher-weight cases above, but are weak signals on their own.

Crucially, the report does not present the score as a black box. For every flagged account, the report breaks the total down — for example, 'Round-Trip Detection (+25) + Structuring/Smurfing (+18) + Near-Zero Dwell Time (+12) = 55' — together with the LLM explanation for each contributing case. This transparency is what allows a defence lawyer or judge to examine exactly how the system reached its conclusion, which is essential for evidence intended for use in court.

5.7 Output of the Analysis Phase

On completion, the analysis phase produces four artefacts that feed into Phase 4 (Output) and the RAG chatbot:

- An enriched transaction dataframe — the original standardised table with additional columns recording which cases flagged each transaction.

- A findings list — every triggered case, per account, with its LLM-written explanation, evidence-strength rating, and recommended action.
- A ranked suspicion-score table — every account ordered by its composite score, with the per-case point breakdown.
- The transaction graph object (NetworkX) — used both for the static image embedded in the PDF report and the interactive graph rendered in the frontend.

6. Report Generation Design

The report-generation phase converts the analysis findings into a professional document that the investigator can download, act on, and submit to court without requiring any further technical interpretation.

6.1 Audience-Based Report Structure

The report is read by three different people, each needing a different level of detail. The report is therefore organised into three corresponding sections:

Reader	What They Need	Report Section
Senior Officer (SP / DCP)	A one-page, plain-language summary of whether the suspect network is suspicious and what the bottom-line recommendation is.	Executive Summary
Investigating Officer	Every flagged transaction, every triggered case, every account involved, with clear plain-English explanations for each.	Detailed Findings
Forensic Expert / Lawyer	The raw mathematical evidence — exact graph cycles, hop counts, statistical scores, timestamps — needed to verify findings and defend them in court.	Technical Appendix

Section 1 — Executive Summary

One page, written in plain language with no technical terms. States how many accounts were analysed, how many were flagged, the highest suspicion score, the total suspicious amount, and the single most critical finding in one or two sentences. For example: 'Analysis of 8 accounts covering 14,000 transactions identified ACC001 as highly suspicious with a score of 87 out of 100. A total of Rs 14,73,000 moved through this account in a pattern consistent with drug-payment aggregation. Immediate action is recommended on ACC001 and ACC009.'

Section 2 — Detailed Findings per Account

One subsection per flagged account, containing: the account holder's name and account number; the suspicion score and its point-by-point breakdown; every detection case that triggered, with its plain-English explanation; the specific suspicious transactions in a table (date, narration, amount, reason flagged); the money-flow chain showing where the money came from and where it went; and the LLM's interpretation of the pattern in the context of the case brief.

Section 3 — Technical Appendix

All raw mathematical evidence: NetworkX cycle paths and hop counts, the full list of flagged transactions with every field, balance-validation results, Isolation Forest anomaly scores, the entity map extracted from narrations, and the timestamps of when the analysis was run. This section is what allows a forensic auditor to independently verify that every finding is a calculated fact, not an opinion.

6.2 Money-Flow Visualisation in the Report

The interactive graph shown on screen cannot be embedded directly into a PDF. For the report, a static image of the same graph is rendered using matplotlib: nodes are account IDs, edges are transactions labelled with amounts, suspicious accounts are highlighted in a distinct colour, and terminal/convergence accounts are marked clearly. This image is embedded in the PDF so the investigator has a complete visual of the network on paper.

6.3 Export Formats

PDF (ReportLab): The formal, court-ready document. Structure: cover page (case name, analysis date, account count, tool name); Page 1 — executive summary; subsequent pages — one subsection per flagged account with score breakdown, case explanations, transaction tables and the money-flow image; technical appendix; and a final disclaimer page stating that the report is generated by an automated analysis tool and that findings should be verified by a qualified forensic auditor before being submitted as primary evidence.

Excel (pandas + openpyxl): The working document for the investigating officer — one sheet per flagged account, with all flagged transactions in sortable, filterable tables. Generated in parallel with the PDF.

Both files are saved to the local outputs folder and made available to the frontend via the /report and /export endpoints.

6.4 Role of the LLM in Reporting

The LLM contributes to the report in exactly two places. First, the per-finding explanations generated during the analysis phase (Section 5.5) are inserted directly into Section 2 of the report. Second, after all findings and scores are collected, the LLM is given the complete findings list once and asked to write the one-page executive summary in language suitable for a senior officer. Every table, score, transaction list and the technical appendix is produced directly by Python and ReportLab from the analysis output — no LLM involvement.

6.5 Implementation Order for Reporting

1. Build the Excel export first — it is pandas dataframes written to sheets, the simplest piece to get working.
2. Build the PDF with ReportLab, starting with just the flagged-transaction tables.
3. Add the per-account score breakdown and case explanations.
4. Add the money-flow image embedding (matplotlib).
5. Add the LLM-written executive summary.
6. Add the technical appendix and the legal disclaimer page.

7. RAG-Based Investigation Chatbot Design

7.1 Purpose

After the report is generated, the investigator inevitably has follow-up questions — 'show me all transactions between ACC001 and ACC009', 'how much did Ramesh receive in February', 'which sender sent the most number of times'. Without a chatbot, answering these requires returning to Excel and manually filtering, which most investigators are not equipped to do quickly. The chatbot lets the investigator ask these questions in plain English and get an answer grounded in the actual uploaded data within seconds.

7.2 RAG Pipeline

Retrieval-Augmented Generation solves a specific problem: with up to 1,00,000 transactions in a case, the system cannot send the entire dataset to the LLM for every question — it would exceed token limits, be slow, and be expensive. Instead, the pipeline below ensures the LLM only ever reads the handful of transactions actually relevant to the question asked.

Step	Process	Technology
1	Ingest — bank statements are parsed and transactions extracted (this is the output of the extraction module, Section 4.2).	OCR + LLM Parser
2	Embed — every transaction is converted to a short descriptive sentence and turned into a vector embedding.	Sentence Transformers
3	Store — embeddings are stored in a local vector database, one collection per case session.	ChromaDB
4	Query — the investigator asks a question in natural language through the chat interface.	Chat Interface
5	Retrieve — the system performs a semantic search to find the most relevant transactions (typically 30–50).	Semantic Search
6	Generate — the LLM reads the retrieved transactions, the question, and the conversation history, and writes a grounded answer.	LLM (Grok / Claude API)

Because every answer is generated only from transactions that ChromaDB actually retrieved from the uploaded case data, the chatbot cannot invent transactions that do not exist — its answers are grounded in the real dataset.

7.3 Query Categories

The chatbot is designed to handle three categories of investigator questions:

Category	Example Questions	How It Is Answered
Filtering & Retrieval	Show me all credits above Rs 10,000. Show me every ATM withdrawal in January.	ChromaDB returns the matching transactions; the LLM presents them clearly.
Calculation & Summary	What is the total amount ACC001 received from all senders? How many unique people sent money to Ramesh?	The LLM calculates or summarises from the relevant transactions ChromaDB retrieved.
Investigation Insight	Does the timing of these transactions suggest coordination? Which account in this network looks most like a mule?	The LLM reasons over the retrieved transactions and the case brief to produce an analytical answer.

7.4 Conversation Context

The conversation history for the session is maintained by FastAPI and sent to the LLM with every new question. This allows the chatbot to resolve follow-up questions correctly — for example, if the investigator asks 'show me Ramesh's transactions in March' and then asks 'now show me only the credits above 3000', the LLM can resolve 'now' as referring to the previous result set because it can see the prior exchange.

7.5 Limitations

- The chatbot cannot discover new fraud patterns that the 25 detection cases did not find — it is a text-reasoning interface, not a graph-computation engine. If round-trip detection found no cycle, asking the chatbot 'is there a round trip here' will not reliably find one either.
- The chatbot is read-only. It cannot modify findings, recalculate scores, or regenerate the report.
- The chatbot only has access to the data uploaded in the current case session — it has no knowledge of other cases or external information.

7.6 Why This Is the Standout Feature

Most teams in the hackathon are expected to build extraction and analysis and stop there. A chatbot that answers natural-language questions over the actual uploaded transaction data — live, in front of the judges — is the moment that most clearly demonstrates the value of combining RAG with an LLM for a law-enforcement tool, and is the feature most likely to differentiate this submission.

8. Technology Stack

The table below consolidates the technology choices discussed throughout this document, mapped to the part of the system each one serves.

Category	Technology	Used For
Frontend	React, TailwindCSS, Axios	Web interface, styling, and API communication with the backend.
Network Graph	Cytoscape.js / D3.js	Interactive money-flow visualisation in the results screen.
Backend Framework	Python, FastAPI, uvicorn	Core application logic and the API layer connecting frontend to backend.
Digital PDF Extraction	pdfplumber	Extracts embedded text from computer-generated PDF statements.
OCR	Tesseract (via pytesseract)	Converts scanned documents and photographed statements into text.
Data Standardisation & Cleaning	pandas	Reads Excel/CSV, standardises columns, validates and cleans the unified transaction table.
Structure ID, Thresholds & Explanations	LLM API (e.g., Claude / Groq / GPT-4)	Identifies column structure during extraction; generates dynamic thresholds, keyword extraction, finding explanations, hypothesis generation and the executive summary during analysis.
Graph Analysis	NetworkX	Round-trip, layering, hub, isolated-cluster and convergence-point detection (Cases 1–5).
Time & Amount Analysis	pandas	Time-based and amount-based detection cases (Cases 6–15).
Counterparty & Narration Analysis	pandas	Counterparty and narration-based detection cases (Cases 16–21).
Statistical Anomaly Detection	scikit-learn (Isolation Forest)	Self-trained anomaly detection across the uploaded dataset (Case 22).
Vector Storage & Embeddings	ChromaDB, sentence-transformers	Stores every transaction as a searchable embedding for the RAG chatbot.
Report Generation	ReportLab, openpyxl, matplotlib	Assembles the PDF and Excel forensic reports, including the rendered network-graph image.
File Storage	Local disk (uploads / outputs / cache folders)	Stores uploaded files, generated reports, and cached LLM responses for the hackathon demo.

9. Data Storage & Data Flow

9.1 Storage Components

The system uses three distinct storage mechanisms, each serving a different purpose. It is important that these are kept conceptually separate, since they are populated and consumed at different points in the pipeline.

Component	Technology	Lifetime	Role
In-Memory Dataframe	pandas (RAM)	Temporary — exists only for the session	The live working table of clean transactions. All 25 detection cases read from and write enrichment columns to this dataframe. It is the input to NetworkX graph construction and to the report generator.
Vector Store	ChromaDB (disk)	Persistent across the session	Each transaction, converted into a descriptive sentence, is stored here as a searchable embedding. Used exclusively by the RAG chatbot — the 25 detection cases never read from ChromaDB.
Local File Storage	Disk folders: uploads / outputs / cache	Persistent	Holds the raw uploaded statements, the generated PDF and Excel reports for download, and cached LLM responses so repeated test runs do not consume API quota.

For the hackathon, local disk storage is sufficient: the investigator and the tool run on the same machine, file sizes are manageable, and there is no need for multi-user access. Section 11 discusses how this would be replaced by cloud storage (e.g., AWS S3) and a managed vector database in a production deployment, with no change to the core logic of the system.

9.2 End-to-End Data Flow

The complete path a case follows through the system, from upload to court-ready output, is as follows:

1. The investigator types the case brief and uploads all suspect statements on the React frontend; both are sent to the FastAPI /upload endpoint.
2. FastAPI saves the files to the uploads folder and the case brief to session memory, then triggers the extraction pipeline.
3. The file-type router (Component 1) sends each file down the correct path: pdfplumber for digital PDFs, Tesseract OCR plus pattern cleaning for images and scanned PDFs, or pandas directly for Excel/CSV.
4. The first 30–40 lines of each document's text are sent to the LLM, which returns the column structure (Component 3); the response is cached.
5. Python standardises every row into the unified transaction table (Component 4) and runs the validation checks (Component 5), separating clean rows from flagged rows.
6. The clean table is stored as an in-memory pandas dataframe (for analysis) and simultaneously written to ChromaDB as searchable vectors (for the chatbot).

7. The LLM reads the case brief and returns dynamic thresholds (Case 23), which are applied to the 21 remaining rule-based cases.
8. All 25 detection cases run across the graph (NetworkX), time/amount/counterparty/narration modules (pandas), the statistical module (scikit-learn), and the LLM-driven cases.
9. Every triggered finding is sent to the LLM, which returns a plain-English explanation, an evidence-strength rating, and a recommended action.
10. Composite suspicion scores are calculated per account from the weighted findings, and accounts are ranked.
11. The LLM writes the executive summary from the complete findings list; ReportLab generates the PDF and openpyxl generates the Excel export, both saved to the outputs folder.
12. FastAPI sends the findings, scores and graph data to the React frontend, which renders the interactive network graph, the score rankings, the findings panel, and the download links.
13. The chatbot panel becomes active. Each investigator question is sent to ChromaDB for semantic retrieval, then to the LLM along with the retrieved transactions and the conversation history, and the answer is displayed in the chat panel.
14. The investigator downloads the PDF report and submits it as part of the case file.

10. Security & Privacy Considerations

Every uploaded bank statement contains sensitive financial data belonging to suspects under active investigation. While the hackathon prototype does not implement the full set of controls below, the design explicitly accounts for them so that the system can be presented as production-aware, which is a point judges from CID Karnataka are likely to probe.

- **Encryption** — in a production deployment, uploaded statements and generated reports would be encrypted at rest, and all traffic between the frontend and backend would be encrypted in transit (HTTPS).
- **Authentication & Authorisation** — access to the tool would require investigator login; only authorised personnel with valid credentials could upload data or view reports.
- **Audit Logging** — every action (upload, analysis run, report download, chatbot question) would be logged with the investigator's ID and a timestamp, for audit purposes.
- **Data Lifecycle** — ChromaDB collections and local files for a case would be cleared once the case is formally closed, rather than persisting indefinitely.
- **Cloud Migration Path** — local file storage would be replaced by encrypted cloud object storage (e.g., AWS S3 or Google Cloud Storage), a change confined entirely to the storage layer with no impact on extraction, analysis or reporting logic.

For the hackathon demonstration itself, none of these controls are implemented — the system runs on a single machine for a live demo. Being able to articulate this gap and its production solution, however, is part of the system design and is expected to be communicated to the jury panel.

11. Scalability & Deployment Considerations

The hackathon prototype runs entirely on a single machine. The design, however, anticipates a production deployment across CID Karnataka's offices, and the architecture has been chosen so that scaling requires configuration changes rather than a redesign.

- **Backend Deployment** — FastAPI would be deployed on a cloud server (or container cluster), allowing multiple investigators across different offices to run analyses concurrently.
- **Session Isolation** — because every case session already has its own dataframe and its own ChromaDB collection (Section 9.1), concurrent sessions from different investigators do not interfere with one another.
- **LLM Concurrency** — the LLM API natively supports concurrent requests, so multiple sessions calling the LLM simultaneously does not require additional design work.
- **Storage Migration** — local folders would be replaced by AWS S3 or Google Cloud Storage for uploads and outputs, and the local ChromaDB instance would be replaced by a managed vector database.
- **No Core Logic Changes** — the extraction, analysis, reporting and chatbot logic described in Sections 4–7 remain unchanged; only the storage and deployment configuration differ between the hackathon demo and a production rollout.

This 'same logic, different configuration' framing is intentionally part of the design narrative: it demonstrates that the team has thought beyond the 24-hour demo to how the system would actually be operated by CID Karnataka.

12. Implementation Roadmap

12.1 Recommended Build Order

The build order below is deliberately structured so that the team always has a working, demonstrable system at every stage — the highest-risk and most complex components (OCR, then the LLM-driven extraction steps) are added last, once the core pipeline is proven on simple inputs.

1. Build Components 4 and 5 of the extraction module first, assuming the input is already a clean CSV. Get standardisation and validation working perfectly on hand-crafted test data.
2. Add Component 2A — pdfplumber-based extraction for digital PDFs — feeding into the already-working standardisation and validation steps.
3. Add Component 3 — integrate the LLM for dynamic column-structure identification, replacing any hardcoded column assumptions, and test against a few different bank-statement formats.
4. Add Component 2B — Tesseract OCR plus the pattern-cleaning pass for images and scanned PDFs. This is the most complex component and is deliberately tackled last.
5. Add Component 1 — the file-type router at the top of the pipeline, so the correct path is chosen automatically based on the uploaded file's type.
6. Wire up ChromaDB ingestion once the clean transaction table is reliably produced, so the RAG chatbot has data to work with.
7. Build the 25 detection cases by category (Section 5.4), starting with the deterministic pandas-based cases, then the NetworkX graph cases, then scikit-learn, and finally the three LLM-driven cases (Category G).
8. Build the report generator following the order in Section 6.5 (Excel first, then PDF section by section).
9. Build the RAG chatbot pipeline (Section 7.2) and the conversation-history handling.
10. Build the React frontend screens (Section 4.1 and Section 3.3) and wire them to the FastAPI endpoints, completing end-to-end integration.

12.2 Mapping to the Hackathon Timeline

The CIDECODE 2026 schedule provides a mentoring phase ahead of the hackathon itself, and this roadmap is designed to make use of both windows:

Mentoring Phase (05–25 June 2026): Finalise this system design document, build and validate the extraction pipeline (steps 1–6 above) against the synthetic dataset, and build the core deterministic detection cases (the bulk of step 7). By the end of this phase, the team should have a working extraction-to-analysis pipeline that produces correct findings on synthetic data, even before the frontend exists.

Hackathon Days (4–5 July 2026): Complete the remaining LLM-driven detection cases, build the report generator and the RAG chatbot, build the frontend, integrate everything end-to-end, and prepare the live demonstration — including the chatbot question-answering moment described in Section 7.6.

12.3 Synthetic Dataset Strategy

No public dataset of real Indian bank statements exists, and CID Karnataka cannot provide real case data for a hackathon. The team will therefore generate a synthetic dataset using a Python script, which is standard practice for fintech hackathons and research prototypes alike.

- Create a set of fake accounts (ACC001, ACC002, ...) with realistic Indian bank narration formats — UPI/PhonePe, NEFT, RTGS, IMPS, ATM WDL — and realistic rupee amounts.

- Deliberately build in known fraud patterns: at least one round-trip cycle, one smurfing pattern (many small credits from many senders), one dormant-account activation, and one hawala matched pair.
- Export the same underlying data in all four required formats: CSV and Excel (direct pandas export), a clean digital PDF (via ReportLab), and a scanned-image version (by converting a PDF page to JPG, optionally with slight blur to simulate a phone photo).
- Use this dataset as the accuracy benchmark: when the tool's 25 cases correctly identify exactly the patterns that were deliberately planted, this becomes the team's evidence of correctness for the jury panel.
- For OCR-specific testing, publicly available sample bank-statement images (e.g., searching for 'SBI bank statement sample' or 'HDFC bank statement format') can be used to validate that the OCR pipeline correctly reads real-world statement layouts, in addition to the synthetic dataset.

13. Team Structure & Roles

Team Survey Corps is structured around the four major subsystems of the platform, mirroring the module breakdown used throughout this document. This division allows each member to develop and test their module independently against the standardised transaction schema (Section 4.2), before full integration.

Member	Primary Role	Responsibilities
Member 1	AI & Data Ingestion	Multi-format parsing (PDF, Excel, CSV, images), OCR integration, LLM-based structure identification and normalisation (Extraction Module, Section 4.2), and the embedding pipeline that feeds ChromaDB.
Member 2	Fraud Detection	Implementation of the amount-based, time-based and counterparty-based detection cases (Categories B, C, D), the hawala/smurfing/round-tripping logic, and the scikit-learn Isolation Forest anomaly module (Category F).
Member 3	Graph & Visualisation	Construction of the transaction graph and the five graph-based detection cases (Category A, Section 5.4), and the interactive fund-flow network visualisation using Cytoscape.js / D3.js.
Member 4	RAG + Frontend	The LangChain/ChromaDB RAG pipeline and conversational chatbot (Section 7), the React frontend screens (Section 4.1), and the forensic PDF/Excel report generator (Section 6).

Although each member owns a primary module, the analysis phase (Section 5) — the 25 detection cases — is the joint responsibility of Members 2 and 3, given its central importance to the project's value proposition, with Member 1 supplying the standardised data they depend on and Member 4 surfacing their output through the report and chatbot.

14. Conclusion

This document defines the system design for the Multi-Accused Cross-Account Investigation Engine, covering the requirements, the three-layer / four-phase architecture, and — in particular detail — the analysis phase that forms the project's intelligence core: 25 detection cases across seven categories, of which 22 are deterministic and 3 are LLM-driven, all guided by an investigator-supplied case brief and explained in plain English for the report.

The design deliberately keeps every numeric finding mathematically reproducible while reserving the LLM for the tasks it performs better than code: reading natural language, generating context-appropriate thresholds, and writing explanations. Combined with the RAG-powered chatbot, this gives the system its core narrative — an investigator-guided analysis platform that turns weeks of manual bank-statement review into a few minutes of automated, explainable, court-ready analysis.

With this Stage 1 design complete, the team can proceed to Stage 2 — extraction module development — following the build order set out in Section 12, using the synthetic dataset described in Section 12.3 as the running accuracy benchmark throughout development.